

Experiment 6

Name: Urvi Shivalkar

PRN: B25ET1196

Class: SY-3(C)

Title: Perform Bubble Sort, Selection Sort, Insertion Sort, Quick Sort, and Merge Sort using C

Code:

```
#include <stdio.h>
void bubble_sort(int arr[], int n);
void insertion_sort(int arr[], int n);
void selection_sort(int arr[], int n);
void quick_sort(int arr[], int low, int high);
int partition(int arr[], int low, int high);
void merge_sort(int arr[], int low, int high);
void merge(int arr[], int low, int mid, int high);
```

```
int main()
{
    int arr[100], n, i, choice;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements:\n", n);
    for(i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }
    printf("\n--- SORTING MENU ---\n");
    printf("1. Bubble Sort\n");
    printf("2. Insertion Sort\n");
    printf("3. Selection Sort\n");
    printf("4. Quick Sort\n");
    printf("5. Merge Sort\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch(choice)
```

```

{
    case 1:
        bubble_sort(arr, n);
        break;

    case 2:
        insertion_sort(arr, n);
        break;

    case 3:
        selection_sort(arr, n);
        break;

    case 4:
        quick_sort(arr, 0, n - 1);
        break;

    case 5:
        merge_sort(arr, 0, n - 1);
        break;

    default:
        printf("Invalid choice\n");
        return 0;
}
printf("\nSorted array:\n");
for(i = 0; i < n; i++)
{
    printf("%d ", arr[i]);
}
return 0;
}

void bubble_sort(int arr[], int n)
{
    int i, j, temp;

    for(i = 0; i < n - 1; i++)

```

```

    {
        for(j = 0; j < n - i - 1; j++)
        {
            if(arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

void insertion_sort(int arr[], int n)
{
    int i, j, temp;
    for(i = 1; i < n; i++)
    {
        temp = arr[i];
        j = i - 1;
        while(j >= 0 && arr[j] > temp)
        {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = temp;
    }
}

void selection_sort(int arr[], int n)
{
    int i, j, min, temp;
    for(i = 0; i < n - 1; i++)
    {
        min = i;
        for(j = i + 1; j < n; j++)
        {
            if(arr[j] < arr[min])
            {

```

```

        min = j;
    }
}
temp = arr[i];
arr[i] = arr[min];
arr[min] = temp;
}
}
void quick_sort(int arr[], int low, int high)
{
    int pivot_position;
    if(low < high)
    {
        pivot_position = partition(arr, low, high);

        quick_sort(arr, low, pivot_position - 1);
        quick_sort(arr, pivot_position + 1, high);
    }
}
int partition(int arr[], int low, int high)
{
    int pivot, i, j, temp;
    pivot = arr[high];
    i = low - 1;
    for(j = low; j < high; j++)
    {
        if(arr[j] < pivot)
        {
            i++;
            temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
}

```

```

    return i + 1;
}
void merge_sort(int arr[], int low, int high)
{
    int mid;
    if(low < high)
    {
        mid = (low + high) / 2;
        merge_sort(arr, low, mid);
        merge_sort(arr, mid + 1, high);
        merge(arr, low, mid, high);
    }
}
void merge(int arr[], int low, int mid, int high)
{
    int i, j, k;
    int n1 = mid - low + 1;
    int n2 = high - mid;
    int left[100], right[100];
    for(i = 0; i < n1; i++)
    {
        left[i] = arr[low + i];
    }
    for(j = 0; j < n2; j++)
    {
        right[j] = arr[mid + 1 + j];
    }
    i = 0;
    j = 0;
    k = low;
    while(i < n1 && j < n2)
    {
        if(left[i] <= right[j])
        {
            arr[k] = left[i];
            i++;
        }
    }
}

```

```

        else
        {
            arr[k] = right[j];
            j++;
        }
        k++;
    }
    while(i < n1)
    {
        arr[k] = left[i];
        i++;
        k++;
    }
    while(j < n2)
    {
        arr[k] = right[j];
        j++;
        k++;
    }
}

```

Output:

Output	Output
<pre> Enter number of elements: 4 Enter 4 elements: 12 34 56 78 --- SORTING MENU --- 1. Bubble Sort 2. Insertion Sort 3. Selection Sort 4. Quick Sort 5. Merge Sort Enter your choice: 2 Sorted array: 12 34 56 78 </pre>	<pre> Enter number of elements: 5 Enter 5 elements: 12 23 34 45 56 --- SORTING MENU --- 1. Bubble Sort 2. Insertion Sort 3. Selection Sort 4. Quick Sort 5. Merge Sort Enter your choice: 5 Sorted array: 12 23 34 45 56 </pre>